

Aggregate initialization in the presence of deleted constructors

Abstract

This paper describes an alternative solution to Timur Doumler's paper for making class types with user-declared constructors non-aggregates. The gist of the difference is that this alternative treats implicitly deleted and explicitly deleted constructors the same way, and provides a different design principle - in the approach described, deleted constructors subtract from an overload set but do not necessarily establish such a set, i.e. they turn off specific forms of aggregate initialization but not all of them. .

Jacksonville 2018 discussion feedback

In Jacksonville, when discussing P0960, the following feedback was given by EWG:

- Incorporate ban brace-initializing an object if it would be plausible that it would use a deleted constructor?
11 | 14 | 5 | 0 | 3

This paper and Timur's paper separate out aggregate initialization in the presence of deleted constructors.

Why another approach?

Timur's approach makes class types with user-declared constructors non-aggregates. This means that

```
struct X1 {int& x;};
```

and

```
struct X2 {int& x; X2() = delete;};
```

behave differently. Both of those types have a deleted default constructor, but even in the new approach, X1 remains an aggregate. X1 can be brace-initialized with a value, X2 cannot.

Another point of contention is the following:

```
struct X3 {int a, b; X3() = delete;};  
X3 x{1,2}; // is this reasonable?
```

I'm not sure we can do much more than guesswork on what does or does not make sense to programmers, but to me, it makes sense that deleting a default constructor deletes initializations that don't provide values, whereas

initializations that do provide values are unaffected by it.

To be fair, there are other examples where it's anything but obvious what is or is not obvious. :) Such as

```
struct X4 {int a, b, c; X4(int, int) = delete;};
X4 x{42}; // ok
X4 x2{}; // ok
X4 x3{1, 2}; // ill-formed
X4 x4{1,2,3}; // ok
```

Timur's proposal makes all those initializations uniformly ill-formed.

The non-obvious part is that user-declaring a non-deleted constructor would make that constructor the only valid initialization signature (except for copy/move, from a single object), but user-declaring a deleted constructor doesn't do that. I admit that the suggested mental model of "deleted constructor subtracts from the overload set but doesn't necessarily establish it" is not the strongest model I've seen. It does seem workable, though. It makes sense to me that the user wanted to remove a two-parameter signature from the set of valid initialization signatures. The rest is convenience; if we believe that it's likely that such selective removal is expected, this is a more convenient approach than saying "if you ever declare a constructor, you will need to declare the full set". We already require so for non-deleted constructors, though.

Pseudo-wording

It's perhaps easier to understand my approach in terms of wording. So, in [dcl.init.list]/3.4, we do something like

Otherwise, if T is an aggregate, constructors are considered. If a viable constructor is found, but initializing T with a constructor would be ill-formed, the program is ill-formed. Otherwise, aggregate initialization is performed.